SAP Databricks

Generated on: 2026-05-02 12:16:09 GMT+0000

SAP Business Data Cloud | SHIP

Public

Original content: https://help.sap.com/docs/SAP_BUSINESS_DATA_CLOUD/3708ee482fc441ef8bb91711b1629109?locale=en-US&state=PRODUCTION&version=SHIP

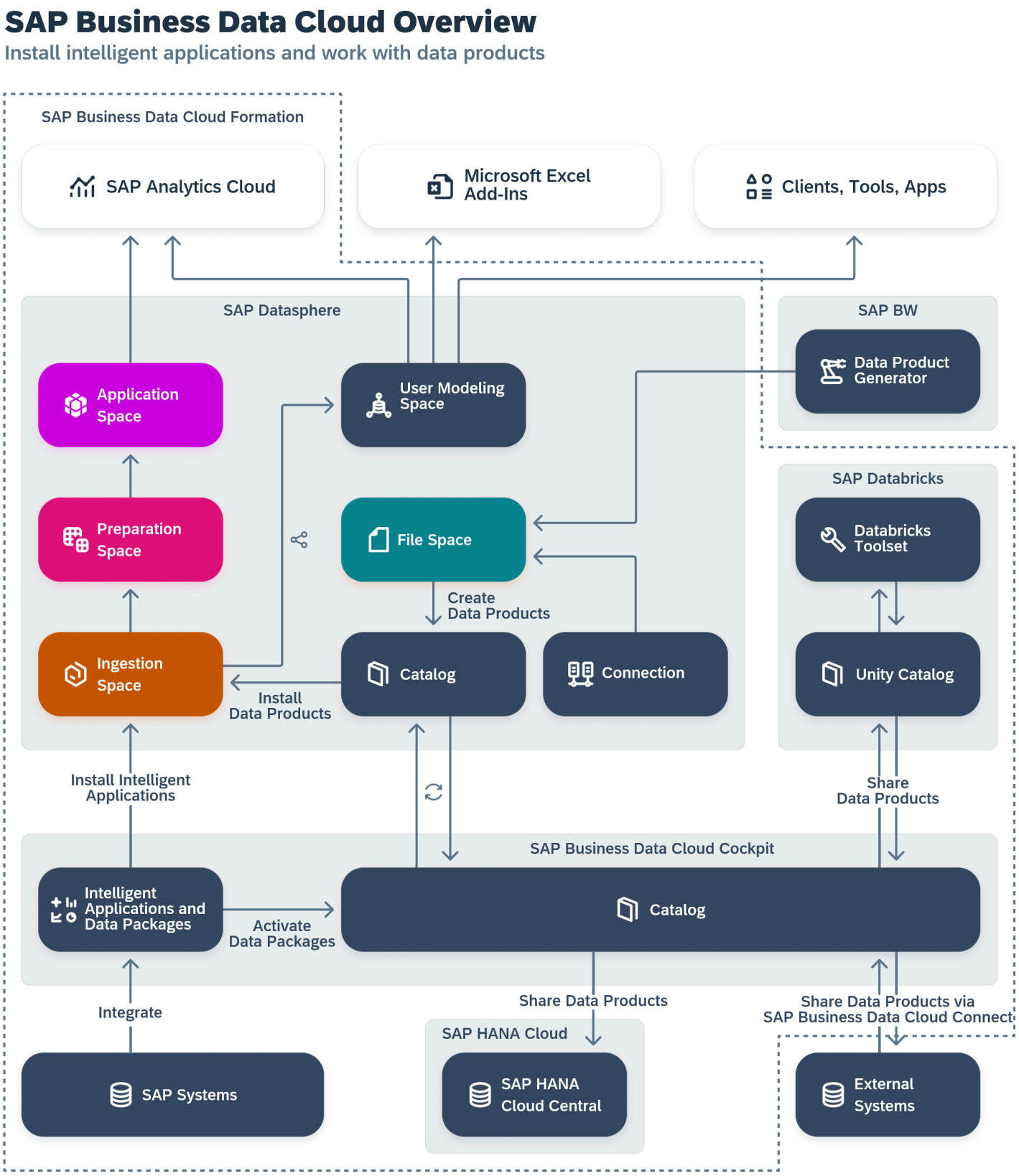
Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

Working with Data Products in SAP Databricks

Users with an appropriate SAP Databricks role can consume, query, process, and transform data products in SAP Databricks, including working with SQL and Python notebooks and using the data products for various data science and AI/ML projects.



To prepare for working with data products in SAP Databricks, a user with an administrator role must:

- Add the SAP Databricks tenant to the formation (see Creating SAP Business Data Cloud Formations).

- Activate one or more data packages (see Activate Data Packages).
- Provide access to the SAP Databricks tenant (see Manage Users and Roles).

Users with an appropriate SAP Databricks role can:

- Learn more about what SAP Databricks is and how it can help you reach your goals (see Introducing SAP Databricks).
- Access data products that are shared from the catalog (see Sharing Data Products to SAP Databricks).
- Install data products directly from the Unity Catalog (see Sharing SAP Business Data Cloud Data Products to SAP Databricks).
- Use the full SAP Databricks toolset to work with your data products (see Get started with Databricks in the *Databricks* documentation).
- Share data products to the SAP Datasphere Catalog (see Sharing Data Products from SAP Databricks to SAP Business Data Cloud).

Adding Privileges into SAP Databricks

You can manage and share data assets and notebooks efficiently in your SAP Databricks environment.

Context

After provisioning SAP Databricks, you must grant users permissions to manage data.

Procedure

1. Access your SAP Databricks Workspace environment. From the sidebar menu, choose **Catalog**.
2. Choose the **Manage** (Gear icon) button and, from the dropdown menu, select **Metastore**.
3. Choose the **Permissions** tab and then choose the **Grant** button.
4. In the **Principals** field, enter the user or a group of users.
5. Select the boxes **CREATE CATALOG**, **CREATE SHARE**, **SET SHARE PERMISSION**, **USE PROVIDER**, **USE RECIPIENT**, and **USE SHARE** then choose **Confirm**.

   You have added the necessary privileges.

Sharing SAP Business Data Cloud Data Products to SAP Databricks

To make data products available for consumption in SAP Databricks, you can either share data products from the SAP Business Data Cloud catalog to SAP Databricks, or in SAP Databricks you can create a catalog from the Delta Share associated with the desired data product.

Sharing Data Products from the SAP Business Data Cloud Catalog to SAP Databricks

Prerequisites

- You have added the necessary privileges. See Adding Privileges into SAP Databricks.

To search for and share data products, you must have a global role that grants you the following privileges:

- **Catalog Asset** (–R––––- -) - To access the catalog and view objects in the **Assets** and **Data Products** collections.
- **Cloud Data Product** (- - - - –S -) - To share data products to target systems.

You can apply the **Catalog Administrator** and the **BD Viewer** roles together, for example, to grant these privileges (see Standard Roles Delivered with SAP Business Data Cloud and Privileges and Permissions in SAP Business Data Cloud).

- A user with an administrator role have activated in SAP Business Data Cloud cockpit the data packages in which these data products are included (see Activating Data Packages).

Context

In the central SAP Business Data Cloud catalog, you can select a data product to share with SAP Databricks.

Procedure

1. In the side navigation area, go to **Catalog & Marketplace** > **Search**.
2. On the catalog search page, use the filters or the search to find the data product you want and select it to open its details page. For more information, see Searching for Data Products and Assets in the Catalog.

   In the **Filter** panel, select the **Data Products** collection and the **Shareable** share status.
3. On the data product details page choose the tab **Overview** > **Details** to review the list of APIs.
4. For the row you want, select the **Share** button to open the **Manage Share Access** dialog.

   → **Tip**

   - If you're still viewing the search results in grid or table view, you can select **Share** for the data product.
   - If you're reviewing the API details page, you can select the **Share** button in the toolbar.

   In the **Overview** section, you can learn more about the data product by review its details and available objects.
5. Under **Target System**, do the following:

   a. If more than one target system is available, select the system. If only one target is available, it's automatically selected.

   b. Enter a name. This name will be the name of the delta share that appears in the target system

   c. You can share the catalog with up to five workspaces. Users in the selected workspaces can access the data product.
6. Select **Share**.

   A message appears letting you know that the share process has started. After the process finishes, a notification appears letting you know the result.

Results

Your data product is shared and its share status is updated and visible in the catalog search results. The name that you entered is the name that users will see in the target system, and it cannot be changed. If you need to change the name, you must delete the share setting for the data product and then share it to the target system again.

In SAP Databricks, users who have authorized access to your data product will be able to utilize the full SAP Databricks toolset to work with it (see Working with Data Products in SAP Databricks).

For information on consuming and processing the data products in SAP Databricks, see Creating and Working with Derived Data Products in SAP Databricks. For information on sharing these data products back to SAP Business Data Cloud, see Sharing Data Products from SAP Databricks to SAP Business Data Cloud.

Creating an SAP Databricks Delta Sharing Catalog from a Data Product

Check SAP Databricks documentation: Create a catalog from a share .

Prerequisites

The user must be a **Data Product Admin** and have the following privileges on the SAP Databricks Metastore: **USE PROVIDER**, **CREATE_CATALOG**. For more details on this role, see Managing SAP Databricks Users and Roles.

Procedure

1. Access the SAP Databricks workspace and open the Catalog Explorer.
2. At the top of the Catalog pane, choose the **Gear** icon and select **Delta Sharing**.
3. On the **Shared with me** tab, find and select the provider corresponding to the data product provider system that is included into the formation, along with your SAP Databricks tenant.

   > **Note**
   > To correlate the providers with the systems in the formation, check the value in the **Comment** column.

4. On the **Shares** tab, find the share and choose **Create catalog**.
   a. Enter a name for the catalog.
   b. Choose **Create**.
5. After the catalog is created, it will be listed under the **Shared** section in the main view of the Catalog Explorer.
6. [OPTIONAL] To grant other users access to read data from this catalog:
   a. Select the catalog name in the Catalog Explorer under the **Shared** section.
   b. Access the **Permissions** tab and choose **Grant**.
   c. Type and select the desired Principal and mark the Privileges: **USE CATALOG**, **BROWSE**, **SELECT**.

For more information, see SAP Databricks documentation: Create a catalog from a share .

Consuming and Processing SAP Business Data Cloud Data Products in SAP Databricks

After a Delta Sharing catalog is created from a data product share, the data can be consumed, queried, processed, transformed by authorized users as a regular catalog in the Unity Catalog. In the next sub-sections, you can find some examples of how to query data using notebooks in SQL and Python. See SAP Databricks documentation for additional information:

- Query data
- Transform data
- Use Delta Lake change data feed

When you share data products from SAP Business Data Cloud to SAP Databricks, you can access their semantic metadata directly in Unity Catalog. For detailed guidance, see SAP Databricks documentation SAP BDC semantic metadata .

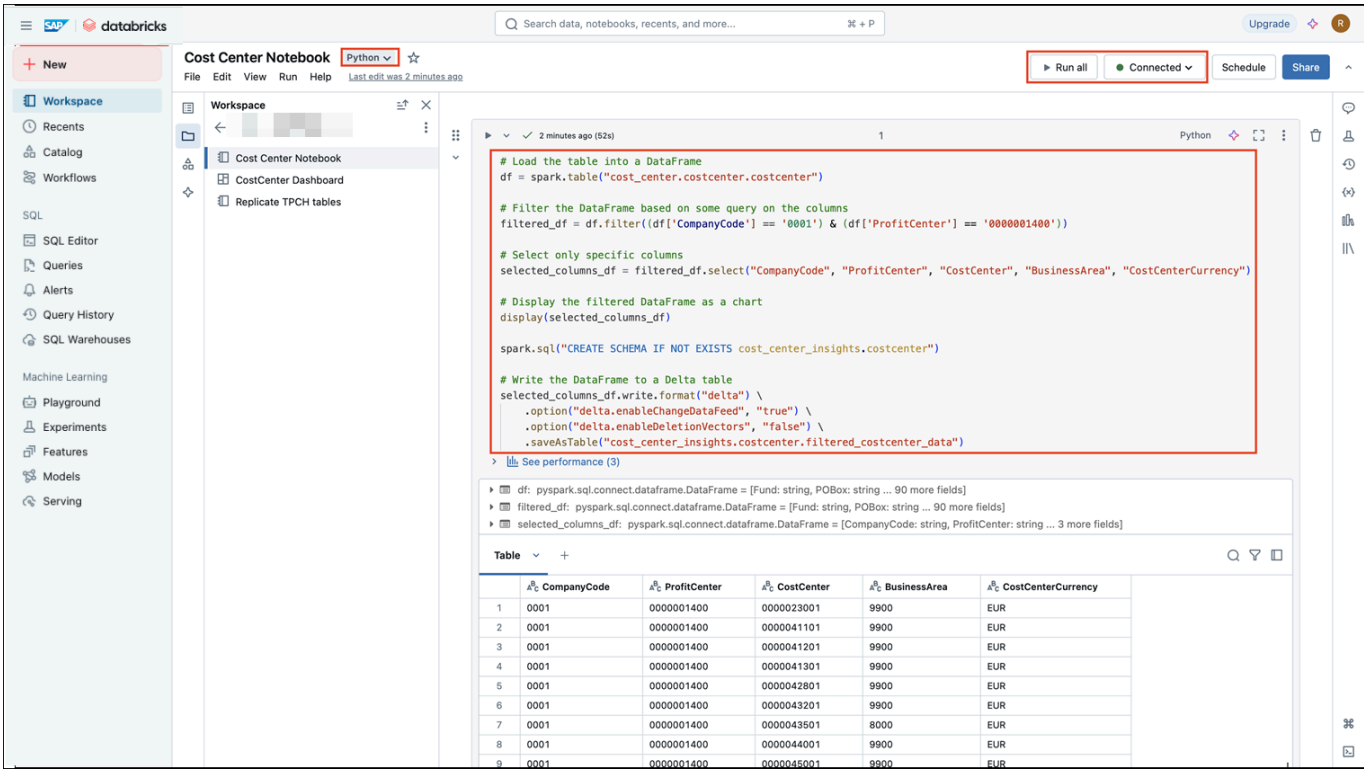
Permissions

To read the data from the SAP data product, the user must have the following privileges on the Delta Sharing catalog associated with the SAP data product (granted by the catalog owner): **USE CATALOG**, **BROWSE**, **SELECT**.

Querying Data in a Python Notebook

Relevant documentation: Get started: Query and visualize data from a notebook .

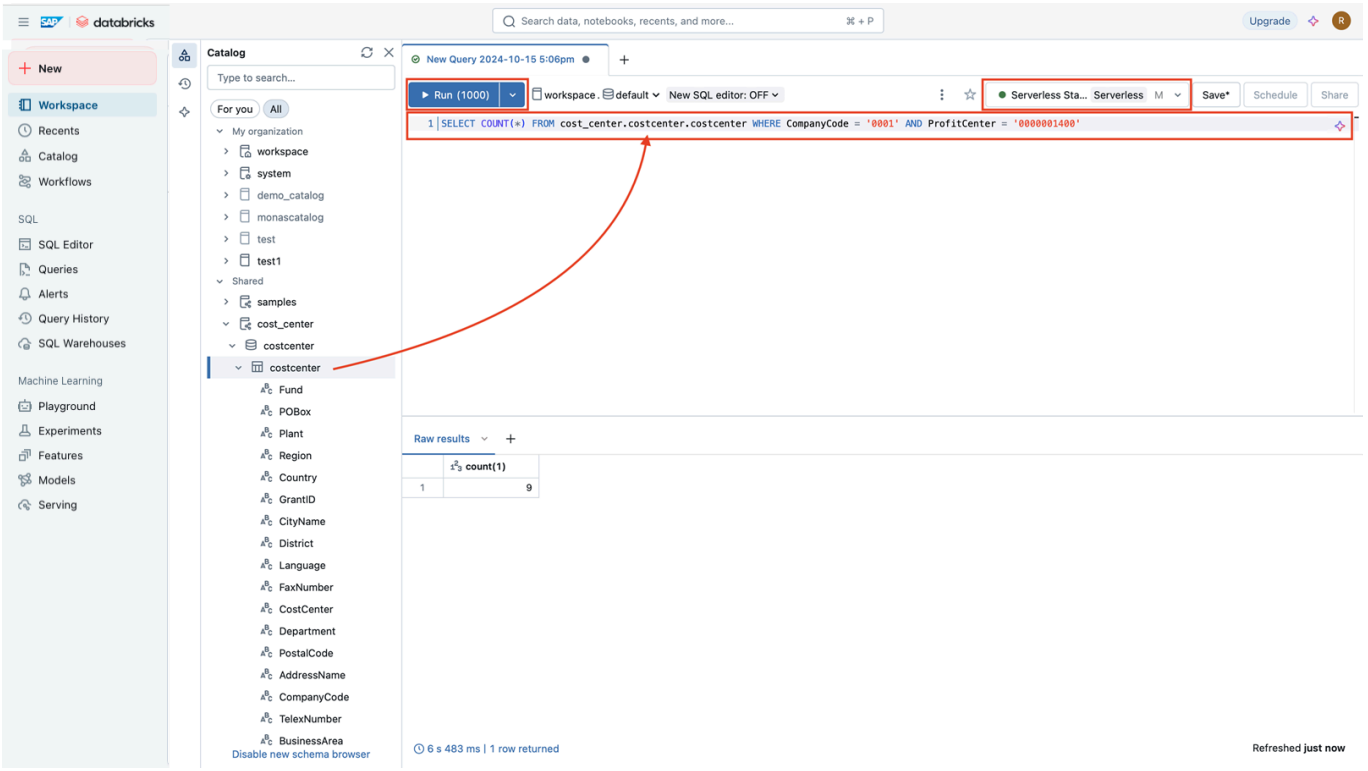
1. Create a new notebook by choosing the **(+) New** button in the sidebar in the top left corner in your SAP Databricks workspace.
2. Select the notebook language on the dropdown menu on the right side of the notebook title.
3. Select the **Serverless** compute on the right side of the **Run All** button on the top right corner.
4. Enter you query in one of the notebook cells and run (choose **Run all**, for example).



Querying Data in the SQL Editor

Relevant documentation: Write queries and explore data in the SQL editor .

1. Access the SQL Editor by choosing the **SQL Editor** button in the sidebar of the SAP Databricks workspace.
2. **Serverless SQL Warehouse** as the compute on the top right corner.
3. Type your query in the main text area.
4. Choose **Run**.



Creating and Working with Derived Data Products in SAP Databricks

The Unity Catalog integration with SAP Business Data Cloud enables bidirectional data flow by storing derived insights as Delta tables with rich ORD and CSN metadata. You can use this to expose SAP Databricks analytics results back to SAP applications like Datasphere for enterprise-wide consumption.

Prerequisites

You have added the necessary privileges. See Adding Privileges into SAP Databricks.

Creating a Catalog in the SAP Databricks Unity Catalog for Writing Derived Data Products

SAP Databricks documentation: Create catalogs .

Permissions

To create a new catalog in the Unity Catalog of SAP Databricks, the user must be a Data Product Admin" and have the following privileges on the SAP Databricks Metastore: USE PROVIDER, CREATE CATALOG.

Procedure

1. In the Catalog Explorer, choose **+** > **Add a catalog**.
2. Type the desired name, choose **Type as Standard**, uncheck **Use default storage** for **Use default storage**, and choose **Create**.
3. Go to the **Permissions** tab after accessing the created catalog and grant (at least) the following privileges to the users that will be able to write to this catalog: **USE CATALOG**, **USE SCHEMA**, **SELECT**, **CREATE SCHEMA** and **CREATE TABLE**.

Writing Data in the Form of Delta Tables to the Unity Catalog

When you create new Delta tables to expose as data products shared via Delta Sharing, you can configure table properties such as deletion vectors and change data feed based on your needs. Change data feed is disabled by default and can be enabled if required.

Permissions

To write the data to a catalog in the Unity Catalog, the user must have the following privileges on this catalog (granted by the catalog owner): USE CATALOG, USE SCHEMA, SELECT, CREATE SCHEMA and CREATE TABLE.

Writing Delta Tables in Python

In Python, the following command can be used to save a DataFrame named `df` as a Delta Table named `cost_center_insights.costcenter.filtered_costcenter_data` with deletion vectors and change data feed enabled:

Sample Code

```
df.write.format("delta") \
    .option("delta.enableChangeDataFeed", "true") \
    .option("delta.enableDeletionVectors", "true") \
    .saveAsTable("cost_center_insights.costcenter.filtered_costcenter_data")
```

Writing Delta Tables in SQL

In SQL, the following command can be used to create a Delta Table named `cost_center_insights.costcenter.filtered_costcenter_data` with deletion vectors and change data feed enabled:

Sample Code

```
CREATE TABLE cost_center_insights.costcenter.filtered_costcenter_data
  TBLPROPERTIES (
    delta.enableChangeDataFeed = true,
    delta.enableDeletionVectors = true
  )
  AS SELECT * FROM cost_center.costcenter.costcenter WHERE CompanyCode = '0001' and ProfitCenter
```

For more information, see Deletion vectors in Databricks and Use Delta Lake change data feed on Databricks.

Share the Delta Tables via Delta Sharing

In this step, a Delta Share will be created from data to be exposed to SAP Business Data Cloud.

Permissions

The user must have the following privileges:

- On the SAP Databricks Metastore: CREATE SHARE, USE RECIPIENT.
- On the catalog containing the data from the data product to be published: SELECT.

Note

Users with the **Data Product Publisher** role already have these permissions.

Procedure

1. From the sidebar menu, choose **Workspace** and, from the **Catalog Explorer**, select the schema or table that you would like to share. Choose the vertical ellipses and select **Share via Delta Sharing**.

   A dialog box appears.

2. Select **Create a new share with the Table/Schema** > Provide a share name, select the **sap-business-data-cloud** Recipient and choose **Share**.

Sharing Data Products from SAP Databricks to SAP Business Data Cloud

This section explains how to use Python SDK for publishing data products in SAP Business Data Cloud so that it becomes discoverable by other consumer applications in SAP Business Data Cloud, for example by SAP Datasphere.

Prerequisites

You have added the necessary privileges. See Adding Privileges into SAP Databricks.

Create a Delta Share

For a Delta Share to be exposed for downstream processing in SAP Databricks, it needs to be published as a data product with rich metadata. This can be done using an iPython Notebook available in your SAP Databricks Workspace. For further details on how to describe the data product, see Appendix A.

1. Access your SAP Databricks Workspace environment.
2. From the sidebar menu, choose **Catalog**.
3. Choose the **Manage** (Gear icon) button and, from the dropdown menu, select **Delta sharing**.
4. Under **Share data**, provide the following information:
   a. **Create share**: In the **Share name** field, enter a share name and choose **Save and continue**. The **Description** field is optional.
   b. **Add data assets**: Select one or more schemas, which includes all the tables in the schema, or one or more individual tables. Choose **Save and continue**.
   c. **Add notebooks**: (Optional) Specify the **Storage location**. Choose **Save and continue**.
   d. **Add recipients**: In the **Recipient** field, from the dropdown menu, select **sap-business-data-cloud**. Choose **Share data**.

ℹ Note

You can share schemas, tables, and views, and generate CSNs for them. Materialized views aren't supported in SAP Databricks.

The Delta Share is now exposed to the workspace.

Publishing the Delta Share as Data Product to SAP Business Data Cloud

1. Create a folder in the workspace by choosing Workspace > Create > Folder. Enter a name for this new folder and choose Create.

2. Create a notebook, in the newly created folder, by selecting that folder and choosing Create > Notebook.

3. Update the serverless environment version by following these steps:

    a. On the sidebar menu, choose the Environment button.

    b. In the Environment version field, use the dropdown menu to choose 3.

    c. Choose Apply.

4. Hover over the area under the rectangle and use +Code to create a cell in the notebook to run the instructions.

5. In this cell, install sap-bdc-connect-sdk:

    a. Execute pip install sap-bdc-connect-sdk to install the python SDK.

    b. Verify the SDK installation by executing pip show sap-bdc-connect-sdk. A successful output should look like this:

Output Code

```
Name: sap-bdc-connect-sdk
Version: 1.0.9
Summary: Python SDK for SAP Business Data Cloud
Home-page: https://www.sap.com
Author: SAP SE
Author-email:
License: SAP DEVELOPER LICENSE AGREEMENT
Location: /local_disk0/.ephemeral_nfs/envs/pythonEnv-738b110e-411e-47f8-bf8e-355d6c2b
Requires: argparse, cryptography, grpcio-status, protobuf, pydantic, python-dateutil,
Required-by:
```

6. Create or update the share in sap-bdc-connect-sdk.

    A share is a mechanism for distributing and accessing data across different systems. Creating or updating a share involves including specific attributes, such as @openResourceDiscoveryV1, in the request body, aligning with the Open Resource Discovery protocol. This procedure ensures that the share is properly structured and described according to specified standards, facilitating effective data sharing and management.

    Copy the below code snippet and paste it to the cell, then provide the share-name, title, shortDescription and description of your share.

Note

Ensure that the shortDescription and description are not the same.

Sample Code

```
from bdc_connect_sdk.auth import BdcConnectClient
from bdc_connect_sdk.auth import DatabricksClient

bdc_connect_client = BdcConnectClient(DatabricksClient(dbutils, "<recipient-name>"))

share_name = "<share-name>"

open_resource_discovery_information = {
    "@openResourceDiscoveryV1": {
        "title": "<title>",
        "shortDescription": "<short-description>",
        "description": "<description>"
    }
}
```

```
bdc_connect_client.create_or_update_share(
    share_name,
    open_resource_discovery_information
)
```

→ Tip

To get the <recipient-name>, go to Databricks Workspace > Catalog > Manage (Gear icon) > Delta Sharing > Shared by me > Recipients . The recipient name is under the field Name.

7. Create or update share CSN.

The CSN serves as a standardized format for configuring and describing shares within a network. To create or update the CSN for a share, it's advised to prepare the CSN content in a separate file and include this content in the request body. This approach ensures accuracy and compliance with the CSN interoperability specifications, facilitating consistent and effective share configuration across systems.

Copy the below code snippet and paste it to the cell, then provide the share-name of your share.

ⓘ Note

The CSN schema can be efficiently generated using a dedicated function generate_csn_template() available the in csn_generator file. This script automates the creation of CSN content for Databricks environments based on a Databricks share.

Sample Code

```
from bdc_connect_sdk.auth import BdcConnectClient
from bdc_connect_sdk.auth import DatabricksClient
from bdc_connect_sdk.utils import csn_generator

bdc_connect_client = BdcConnectClient(DatabricksClient(dbutils, "<recipient-name>"))

share_name = "<share-name>"

csn_schema = csn_generator.generate_csn_template(share_name)

bdc_connect_client.create_or_update_share_csn(
    share_name,
    csn_schema
)
```

→ Tip

To get the <recipient-name>, go to Databricks Workspace > Catalog > Manage (Gear icon) > Delta Sharing > Shared by me > Recipients . The recipient name is under the field Name.

If the generated CSN needs to be modified for any reason, it can be written to a file for editing, for example.

Sample Code

```
from bdc_connect_sdk.utils import csn_generator
import json

share_name = "<share-name>"
csn_schema = csn_generator.generate_csn_template(share_name)

file_path = f"csn_{share_name}.json"
with open(file_path, "w") as csn_file:
    csn_file.write(json.dumps(csn_schema, indent=2))

# change the file
```

```
with open(file_path, "r") as csn_file:
    changed_csn_schema = csn_file.read()

csn_schema = changed_csn_schema
```

8. Publish the data product.

   A Data Product is an abstraction that represents a type of data or data set within a system, facilitating easier management and sharing across different platforms. It bundles resources or API endpoints to enable efficient data access and utilization by integrated systems. Publishing a Data Product allows these systems to access and consume the data, ensuring seamless communication and resource sharing.

   Copy the below code snippet and paste it to the cell, then provide the **share-name** of your share. This will publish the share as a Data Product for consumption by other SAP services in Business Data Cloud.

   ### Sample Code

   ```
   from bdc_connect_sdk.auth import BdcConnectClient
   from bdc_connect_sdk.auth import DatabricksClient

   bdc_connect_client = BdcConnectClient(DatabricksClient(dbutils, "<recipient-name>"))

   share_name = "<share-name>"

   bdc_connect_client.publish_data_product(
       share_name
   )
   ```

   ### Tip

   To get the <recipient-name>, go to **Databricks Workspace** > **Catalog** > **Manage (Gear icon)** > **Delta Sharing** > **Shared by me** > **Recipients** >. The recipient name is under the field **Name**.

   ### Note

   The published data product will not appear instantly in SAP Datasphere, since it triggers a list of actions that take place in pre-scheduled intervals. If you don't see your data products, wait for a few minutes and try again.

Unpublishing a Data Product

Unpublishing a Data Product involves withdrawing access to the data, typically when it has been sufficiently consumed or is no longer desired to be shared. This action helps maintain control over data accessibility and ensures that sharing aligns with security and usage policies. By unpublishing, the system can effectively manage the lifecycle of data and its availability to other integrated platforms.

Copy the below code snippet and paste it to the cell, then provide the **share-name** of your share.

Sample Code

```
from bdc_connect_sdk.auth import BdcConnectClient
from bdc_connect_sdk.auth import DatabricksClient

bdc_connect_client = BdcConnectClient(DatabricksClient(dbutils, "<recipient-name>"))

share_name = "<share-name>"

bdc_connect_client.delete_share(
```

```
    share_name
)
```

→ Tip

To get the <recipient-name>, go to **Databricks Workspace** > **Catalog** > **Manage (Gear icon)** > **Delta Sharing** > **Shared by me** > **Recipients** > . The recipient name is under the field **Name**.

Next Steps

Once a data product has been published from SAP Databricks to SAP Business Data Cloud, you can use the catalog **Data Product** collection to view data products for use in your modeling and other projects. For more information, see Installing Data Products.

Appendix A. Describing the data product with ORD and CSN Metadata

A data product needs to be described in ORD (Open Resource Discovery) and CSN (Core Schema Notation), compliant to CSN Interop Specification v1.0 https://sap.github.io/csn-interop-specification/ .

The ORD information allows you to describe the content of the data product using text, links, tags, labels, etc. The CSN definition intends to describe the schema of the data, specifying the tables being shared, and their columns with the definition of data types, primary keys, associations/references, etc.

ORD Data Product definition

The ORD data product schema contains the complete list of attributes that compose the ORD data product metadata. However, not all fields can be set by users. For more details, check the following links:

- Fields that the user MUST provide:

  These are the fields that are mandatory in the data product object and cannot be automatically generated by the provider system. Therefore, they must be provided by the user publishing the data product.

Field	Mandatory	Example Value
title	yes	**"Customer order"**
shortDescription	yes	**"Offering access to all online and offline orders submitted by customers."**
description	yes	**"The data product Customer Order offers access to all online and offline orders submitted by customers. It provides a customer view on the orders. For fulfillment-specific aspects please refer to the data product Fulfillment Order."**

- Fields that the user can provide:

  These are the fields that are optional in the data product object and will not be automatically generated by the provider system. By default, they will not be present in the resulting data product unless defined by the user.

Field	Mandatory	Example value	Comments

labels	false	**Sample Code**
		`{`
		`"label-key-1": [`
			Use to indicate reserved label keys that cannot

Field	Mandatory	Example value	Comments
		"label-value-1", "label-value-2"] }	be overriden by the user.
deprecationDate	false	Sample Code "2025-12-19T15:47:04+00:00"	
sunsetDate	false	Sample Code "2026-12-19T15:47:04+00:00"	
successors	false	Sample Code ["customer.ext:dataProduct:other:v1"]	
industry	false	Sample Code ["Retail", "Public Sector"]	
lineOfBusiness	false	Sample Code ["Sales"]	
tags	false	Sample Code ["storage", "high-availability"]	
correlationIds	false	Sample Code ["sap.s4:communicationScenario:SAP_COM_0008"]	

dataProductLinks	false	Sample Code
		[
		{
		"type": "support",
		"url": "https://example.com/"
		}
		]	

links	false	Sample Code	

Field	Mandatory	Example value	Comments

		<pre>[
		  {
		  "title": "Example Link",
		  "url": "https://example.com/",
		  "description": "My example link description"
		  }
		]</pre>	

documentationLabels	false	**Sample Code**
		<pre>{
		  "My Documentation Label": [
		  "with link"
		  ]
		}</pre>	

- Fields that can be overwritten by the user:

Field	Mandatory	Example value	Comments
visibility	yes	"public"	Default value: "public". Alternative values: "interval", "private".
releaseStatus	yes	"active"	Default value: "active". Alternative values: "beta", "deprecated".

The minimum required fields that need to be specified are title , shortDescription and description .

Note

The **description** value must not contain the **shortDescription** value.

You can describe the ORD attributes for your data product on a JSON document. This JSON document needs to be created or uploaded to your SAP Databricks workspace (see Workspace files basic usage).

See an example of how the ORD information JSON document should look like:

Sample Code

```
{
  "title": "TPC-H",
  "shortDescription": "TPC-H is a decision support benchmark.",
  "description": "TPC-H consists of a suite of business-oriented ad hoc queries and concurrent d
}
```

CSN Interop Definition

In addition to the ORD metadata, the CSN Interop metadata is also required when publishing a data product. For a complete documentation of the CSN Interop specification, check CSN Interop Effective v1 • Interface Documentation . For examples of CSN documents compliant to this specification, check CSN Interop Effective v1 • Examples .